

Learning to Evict: A Neural Buffer Replacement Policy for BusTub

1st Zhicheng Zhou

CS Department

University of Maryland - College Park

College Park, MD

zzhou121@umd.edu

1st Akhil Gunda

CS Department

University of Maryland - College Park

College Park, MD

agunda@umd.edu

Abstract—Buffer pool replacement policies in modern database management systems rely on static heuristics, such as LRU and Clock, which cannot adapt to the varying access patterns of different workloads. Recent work has shown that machine learning can approximate Bélady’s optimal replacement algorithm in simulations, but practical deployment inside a real DBMS buffer pool remains largely unexplored. In this paper, we present LEARNEDREPLACER, a learned buffer replacement policy implemented inside CMU’s BusTub DBMS. We formulate eviction as a regression problem over reuse distance (the number of operations until a frame is next accessed). This directly approximates Bélady’s policy of evicting the frame needed furthest in the future. Our model, a small feedforward neural network, is trained offline in PyTorch, exported to ONNX, and executed at runtime in C++. This creates an end-to-end inference pipeline with zero Python dependency. Evaluating across sequential scan, uniform random, and 80/20 hot-set workloads, LEARNEDREPLACER consistently matches or outperforms LRU. Notably, it achieves 86% of the theoretical optimum hit rate on sequential scans, where LRU achieves 0% by design. Through four iterations of refinement, we share three broader system insights: distribution alignment dominates model architecture choices; predicting continuous reuse distance outperforms binary hot/cold classification; and the C++/Python ONNX boundary requires careful normalization handoffs. The full source code and training trace for LEARNEDREPLACER are publicly available in our GitHub repository¹.

I. INTRODUCTION

The buffer pool’s replacement policy dictates database performance by deciding which pages remain in memory and which are evicted to disk. For decades, Least Recently Used (LRU) and Clock have been the standard heuristics for buffer management. However, both suffer from strict limitations. LRU is famously vulnerable to *sequential flooding*: a table scan larger than the buffer pool flushes out genuinely valuable pages, dropping the cache hit rate to zero for the scan and degrading performance for concurrent queries. Furthermore, exact LRU imposes high implementation overhead, requiring locks and constant-time updates to data structures on the hot path. While the Clock policy mitigates this overhead by approximating LRU, it remains equally susceptible to sequential flooding.

Fundamentally, LRU and Clock are static heuristics unable to adapt to varying access patterns. Recent research demonstrates that machine learning can overcome this by approximating Bélady’s offline optimal algorithm [1], which evicts the page needed furthest in the future. While prior learned models (e.g., LBR [2], LeCaR [3], PARROT [4], LSTM-CRP [5]) show promise, they are almost exclusively evaluated in standalone cache simulators rather than integrated into a real DBMS buffer pool.

In this paper, we close that gap with LEARNEDREPLACER, an end-to-end learned buffer replacement policy integrated directly into CMU’s BusTub DBMS [6]. We formulate page eviction as a regression problem: a small feedforward neural network predicts the *reuse distance* of each frame, evicting the one whose next access is predicted to be furthest in the future. The model is trained offline in PyTorch using four engineered features (**access frequency**, **time since last access**, **average inter-access interval**, and **access type**) plus a workload-type indicator. It is then exported to ONNX [7] and executed at runtime in C++ via ONNX Runtime, entirely removing Python dependencies from the database engine.

We evaluate LEARNEDREPLACER against LRU across sequential scan, uniform random, and 80/20 hot-set workloads (10,000 operations on a 64-frame buffer pool). Our model matches or exceeds LRU performance across the board and most notably achieving 86% of the theoretical optimum hit rate on sequential scans, where LRU achieves 0% by design. Finally, alongside our numerical results, we share practical lessons learned from our four-iteration development process on deploying ML models inside a C++ system architecture.

Our main contributions are:

- An end-to-end implementation of learned buffer replacement inside a real DBMS buffer pool, not a simulator.
- A reuse-distance regression formulation that directly approximates Bélady’s optimal algorithm and generalizes across workload types via a workload-identifier feature.
- A practical deployment pipeline (PyTorch → ONNX → C++) that requires zero Python at runtime.
- An empirical evaluation across three workloads, plus a candid iteration log identifying distribution alignment, class imbalance, and the C++/Python normalization hand-off as the highest-leverage engineering concerns.

¹<https://github.com/ZhichengZhou-1/bustub-learned-replacer/tree/learned-replacer>

The remainder of this paper is organized as follows. Section II reviews buffer management and Bélady’s optimal algorithm. Section III surveys related work on learned cache replacement. Section IV presents the design of LEARNEDREPLACER, and Section V describes its implementation inside BusTub. Section VI reports our experimental results. Sections VII and VIII discuss findings and limitations, and Section IX concludes.

II. BACKGROUND

This section reviews the buffer pool’s role in a DBMS, the two classical replacement policies relevant to our work (LRU and Clock), and the theoretical optimum (Bélady’s algorithm) that any practical policy is implicitly trying to approximate.

A. The Buffer Pool

A DBMS buffer pool is an in-memory cache of disk pages [8]. When a query needs to read or write a page, it requests that page from the buffer pool manager, which either returns the page directly (a *hit*) or fetches it from disk (a *miss*). Because disk I/O is several orders of magnitude slower than memory access, hit rate is the dominant factor in query performance. The buffer pool is finite; when it fills up and a new page must be brought in, the buffer pool manager must choose an existing page to evict. The component that makes this choice is the *replacement policy*, and it is the focus of this paper. A replacement policy operates on *frames* which is the fixed-size slots in the buffer pool that hold pages. At any moment, each frame is either pinned (in active use by a query therefore cannot be evicted) or evictable. When eviction is required, the policy selects one evictable frame to discard. The quality of a policy is measured by the resulting hit rate over a workload: the fraction of page requests served from memory rather than disk.

B. Least Recently Used (LRU)

LRU is the canonical baseline replacement policy [9]. The intuition is simple: a page that was used recently is likely to be used again soon, so the safest page to evict is the one that has not been touched for the longest time. To implement LRU faithfully, the buffer pool manager must maintain the access order of every resident page. The standard implementation uses a doubly-linked list ordered from least to most recently used, paired with a hash table mapping page identifiers to their list nodes. On each access, the relevant node is unlinked and moved to the most-recently-used end; on eviction, the head of the list is removed. Both operations are $O(1)$ amortized. LRU is effective on workloads with strong temporal locality when recently accessed pages are likely to be accessed again. Online transaction processing workloads, which repeatedly touch a small working set of hot pages, fit this model well, and LRU performs near-optimally on them. LRU has two well-known weaknesses. The first is *sequential flooding*. When a query scans a table whose working set exceeds the size of the buffer pool, every page touched by the scan becomes the most recently used, displacing genuinely valuable pages from

other queries. After the scan completes, the buffer pool is filled with pages that will not be touched again, and queries that previously enjoyed cache hits now miss. In the worst case, a sequential scan whose working set strictly exceeds the cache and which loops repeatedly which cause LRU’s hit rate fall to exactly zero, because the least recently used page on every access is precisely the page about to be requested next. The second weakness is implementation cost. The bookkeeping required to maintain exact LRU ordering touches the hot path of every page access. Each request to the buffer pool must update the linked list, which requires acquiring a latch in a multi-threaded environment. The latch becomes a serialization point, and the work itself adds to the latency of every page reference. In high-throughput systems, the cost of maintaining true LRU can rival the savings it produces.

C. Clock

The Clock replacement policy [10] was developed largely to sidestep LRU’s implementation overhead. Rather than maintaining a precise access order, Clock approximates LRU using a single *reference bit* per page and a circular sweep. Each page has a reference bit that is set when the page is accessed. The replacer keeps a clock hand pointing at one of the frames. When eviction is required, the hand advances around the circular buffer of frames. If the current frame’s reference bit is set, the bit is cleared and the hand advances. If the bit is unset, that frame is evicted and the hand stops at the next position. In effect, a page survives one sweep of the clock hand if it has been touched since the last sweep, and is evicted if it has not. The advantages of Clock over LRU are practical: the per-access bookkeeping is a single bit-set rather than a list update, the data structures are simpler and cheaper to make thread-safe, and the sweep amortizes the eviction work over many accesses. For these reasons Clock and its variants are the policy of choice in production database systems and operating systems where LRU’s cost is prohibitive. The disadvantages are that Clock is only an approximation of LRU. Because the reference bit is binary, Clock cannot distinguish between a page accessed once long ago and one accessed many times recently. It also remains vulnerable to sequential flooding: a scan still sets the reference bit on every page it touches, so the scanned pages still survive at least one sweep before becoming candidates for eviction.

D. DBMIN

Unlike LRU and Clock, which are *workload-agnostic* and rely solely on observed access history, DBMIN [8] is a *workload-aware* policy that leverages query plan information directly to guide eviction. The core insight of DBMIN is that database operators exhibit predictable access patterns. For example, sequential scans iterate once with no reuse, whereas index lookups repeatedly hit a small working set. DBMIN exploits this by partitioning the buffer pool into *locality sets* allocated per operator, applying a specific replacement strategy to each. A sequential scan is restricted to a small “use once and evict” pool, which structurally prevents sequential flooding.

While highly effective, this approach requires tight, restrictive integration with the query planner to obtain explicit operator-level metadata. DBMIN is conceptually foundational to our work because it demonstrates that workload-aware buffer management can substantially outperform static heuristics. However, where DBMIN extracts this knowledge at compile time from the query planner, our learned approach dynamically infers it at runtime. By incorporating workload-type indicators as model features, our approach achieves the performance benefits of workload-awareness therefore preventing issues like sequential flooding without requiring tightly coupled planner metadata.

E. *Bélady’s Optimal Algorithm*

In 1966, Bélady proved that the theoretically optimal page replacement policy is to evict, on each eviction event, the page whose next reference is furthest in the future [1]. Intuitively, this is the page the system can most afford to lose. No online policy can do better in terms of hit rate. Bélady’s algorithm cannot be implemented in practice because it requires knowing the entire future access sequence at the moment of each eviction. It is nonetheless invaluable as an upper bound: every practical policy can be evaluated against Bélady’s optimum on a fixed trace, and the gap between them represents room for improvement. A central premise of recent learned-replacement research [3], [4], [11], [12] is that this gap can be narrowed by training a model to *predict* which page will be accessed furthest in the future thus approximating Bélady’s oracle from observed history. Our work follows this approach, **formulating eviction as a regression problem over predicted reuse distance and selecting for eviction the frame whose next access is predicted to be furthest away.**

F. *Summary*

LRU is conceptually clean and effective on workloads with temporal locality, but suffers from sequential flooding and implementation overhead. Clock trades exactness for cheaper bookkeeping and is widely deployed in practice, but inherits LRU’s vulnerability to sequential scans. Both are static heuristics that cannot adapt to the workload at hand. Bélady’s optimal algorithm provides the theoretical upper bound but is unimplementable online. The remainder of this paper describes how a small learned model, trained offline on access traces and deployed via ONNX [7], can approximate Bélady’s policy inside a real DBMS buffer pool [6] and how, on workloads where LRU systematically fails, the resulting policy gets meaningfully closer to optimal.

III. RELATED WORK

Buffer replacement policy research spans over four decades and falls into three broad categories: classical heuristics, workload-aware heuristics, and more recently, machine-learned policies. We survey each in turn, then highlight the gap that LEARNEDREPLACER addresses.

A. *Classical Heuristics: Recency, Frequency, and Hybrids*

The foundational policies (LRU and Clock) are described in detail in Section II. Their core limitation is that their decision rules are fixed at design time and cannot adapt to the workload. Two classical algorithms attempt to overcome this through more expressive bookkeeping.

LRU-K. O’Neil, O’Neil, and Weikum proposed LRU-K [13], which evicts the page whose K -th *most recent* access is furthest in the past, rather than the single most recent access. With $K=2$, a page touched only once (as in a sequential scan) has an infinite backward-2 distance and becomes an immediate eviction candidate. This gives LRU-K robust scan resistance that plain LRU cannot achieve. However, maintaining the K -th access timestamp for every page requires $O(\log N)$ priority-queue operations per eviction, which adds overhead on the critical path of high-throughput systems [13].

ARC. Megiddo and Modha’s Adaptive Replacement Cache (ARC) [14] eliminates the need for manual parameter tuning by dynamically balancing a recency list (T_1) and a frequency list (T_2) using two ghost caches (B_1 , B_2) of recently evicted pages. When a page is found in a ghost cache, ARC adjusts the target sizes of T_1 and T_2 to favor whichever eviction strategy would have retained it. This self-tuning makes ARC empirically near-optimal across a wide range of access patterns, and it is widely deployed in production storage systems. BusTub’s actual default replacer is an ARC variant [14], making it a stronger baseline than the LRU we evaluate against, a limitation we acknowledge in Section VIII.

Despite their sophistication, LRU-K and ARC are still static in that their policy logic is hand-crafted and cannot be retrained when workload characteristics shift.

B. *Workload-Aware Heuristics: DBMIN*

DBMIN [8] (Chou and DeWitt, 1985) demonstrated that explicitly incorporating query-plan semantics into buffer management dramatically outperforms workload-agnostic heuristics. DBMIN partitions the buffer pool into per-operator *locality sets* and applies a different replacement strategy to each: sequential scan operators are assigned a tiny use-once pool that structurally prevents sequential flooding, while index lookups receive a larger retained set to exploit their strong temporal locality. The key limitation of DBMIN is tight coupling: it requires the query planner to hand off operator-level metadata at compile time, making it difficult to deploy in systems where access patterns are not fully predictable from the plan. LEARNEDREPLACER is philosophically aligned with DBMIN in that workload awareness is central to both, but it achieves this awareness through a learned `workload_feat` indicator rather than explicit planner integration.

C. *Learned Cache Replacement*

The past decade has produced a rich body of work applying machine learning to cache and buffer replacement. These approaches share the goal of approximating Bélady’s offline optimal policy from observed access history [3], [4], [11], [12].

LeCaR. Vietri et al. introduced LeCaR (Learning Cache Replacement) [3], which applies *regret minimization* to dynamically choose between LRU and LFU. LeCaR maintains a probability distribution over these two expert policies and updates it based on regret: whenever a missed page is found in the eviction history of one policy, that policy is penalized. LeCaR demonstrated up to an $18\times$ improvement in hit rate over ARC in small-cache scenarios. CACHEUS [15] subsequently extended this framework with additional expert strategies targeting scan-heavy and churn-heavy workloads. Both operate as standalone cache simulators with no integration into a live database engine.

PARROT. Liu et al. framed cache replacement as an *imitation learning* problem [4]. PARROT trains a recurrent neural network to imitate B  lady’s oracle, learning eviction decisions from past access sequences without access to future information. Evaluated on 13 memory-intensive SPEC applications, PARROT reduced cache miss rates by 20% over prior state-of-the-art methods, and increased cache hit rates by 61% over LRU on large-scale web search benchmarks. Critically, PARROT targets CPU-level L2/L3 cache lines rather than disk-page buffer management, and its evaluation is trace-driven simulation rather than deployment inside a DBMS.

LBR. Yuan and Jin proposed the Learned Buffer Replacement (LBR) policy [2], the prior work most closely related to ours. LBR trains both a classifier and a regressor over historical database access patterns to predict future page reuse, with the specific goal of improving DBMS buffer pool performance. However, LBR is evaluated using offline trace replay rather than integration into a live DBMS. LEARNEDREPLACER extends this line of work by embedding the trained model directly inside BusTub’s BufferPoolManager, so the learned policy interacts with real query execution rather than a recording of past accesses. This exposes practical challenges that trace replay conceals, including distribution shift, the C++/Python normalization boundary, and safe LRU fallback.

LSTM-CRP. Closer to hardware, LSTM-CRP [5] presents an algorithm-hardware co-design that uses multiple lightweight LSTMs implemented on FPGA to classify memory access patterns (streaming, thrashing, friendly, mixed) and select the best predictor at runtime. LSTM-CRP improved average cache hit rates by 20% over LRU while operating at 200 MHz with only 2.74 W of power consumption. Its target is CPU-cache replacement on FPGA hardware, a fundamentally different deployment context from a software DBMS buffer pool.

HALP. At the largest scale, Song et al. deployed HALP [16] (Heuristic Aided Learned Preference) on YouTube’s content delivery network. HALP trains a small two-layer MLP reward model continuously and online using pairwise preference comparisons derived automatically from B  lady’s optimality structure, then uses randomized candidate sampling from a fast LRU heuristic to make eviction decisions efficiently. In production, HALP reduced YouTube’s CDN byte miss ratio during peak by an average of 9.1% while incurring only 1.8% CPU overhead. HALP operates as a CDN object cache, not

a relational DBMS buffer pool, and its online training loop assumes a high-throughput, long-running CDN environment rather than a transactional database engine.

D. The Integration Gap

A consistent pattern across all prior learned replacement policies is their evaluation environment: standalone simulators, CPU L2/L3 cache hardware, or CDN object caches. To our knowledge, no prior work has integrated a trained neural network as the primary eviction policy inside the buffer pool of a relational DBMS, operating on real query-generated page requests rather than replayed traces. This integration introduces engineering challenges that simulation cannot expose: normalization handoffs across the C++/Python deployment boundary, safe LRU fallback guarantees when the model is unavailable, and covariate shift between LRU-generated training traces and the policy’s own inference-time access distribution. LEARNEDREPLACER addresses all three challenges and provides, to our knowledge, the first empirical evaluation of learned buffer replacement inside a real DBMS buffer pool.

IV. DESIGN

This section presents the high-level architecture of LEARNEDREPLACER. We focus on system design and rationale, deferring code-level implementation details to Section V.

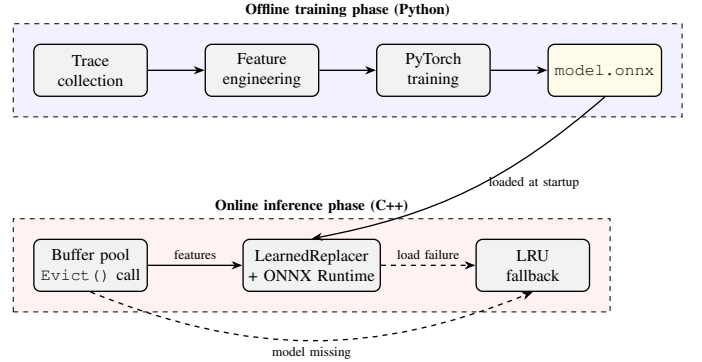


Fig. 1. System architecture of LEARNEDREPLACER. The design temporally separates the offline Python training environment (top) from the performance-critical C++ inference engine (bottom). If the ONNX model fails to load or throws a runtime exception, the system safely routes to a baseline LRU policy.

A. System Overview

Figure 1 shows the overall pipeline, which operates in two disjoint phases.

The *training phase* runs offline in Python. We collect page access traces under representative workloads, engineer per-frame features, generate regression labels, and train a feedforward neural network that is exported as a standalone ONNX file.

The *inference phase* runs online in C++. The exported model is loaded once at database startup. Whenever an eviction is required, the replacer evaluates its evictable frames, computes the required features, runs a forward pass, and evicts the frame predicted to be the safest to discard.

B. Formulating Eviction as Reuse-Distance Regression

The core design decision is the model’s output formulation. We formulate eviction as *regression over reuse distance*.

Instead of treating eviction as a binary classification problem (e.g., predicting if a page will be accessed “within N operations,” which discards vital temporal ordering), our model predicts a continuous value: the exact number of operations until a frame is next accessed. The policy then evicts the frame with the largest predicted distance.

This is a direct approximation of Bélady’s optimal algorithm. Where Bélady’s looks up the true reuse distance from an oracle, our model predicts it from observable history. Because reuse-distance distributions possess long tails that can destabilize training, we train the network using Huber loss rather than mean squared error.

C. Features

The model relies on five engineered features. All features are normalized using statistics computed during training; crucially, these exact statistics are baked into the C++ inference path to prevent distribution shift at runtime.

- **Frequency:** Cumulative accesses to the frame, capturing LFU-style signals.
- **Time since last access:** Operations since the last touch, capturing LRU-style signals via a stable rolling window.
- **Average inter-access interval:** The running mean of gaps between accesses, capturing the rhythm of page usage.
- **Access type:** BusTub-specific hints (`Lookup`, `Scan`, `Index`) that allow the model to distinguish repeating queries from sequential floods.
- **Workload feature:** An integer encoding the current workload type (sequential, random, mixed). This allows a single model to dynamically specialize rather than collapsing to a compromised, workload-averaged policy.

D. Pipeline Stages

The end-to-end design operates in five sequential stages:

- 1) **Trace Collection:** The database, running a baseline LRU policy, logs every page access (timestamp, frame, page, type) to generate workload-specific traces.
- 2) **Feature Engineering:** We compute the rolling features using only historically available data at the time of each access, strictly per-workload to avoid contaminating histories.
- 3) **Label Generation:** Ground-truth reuse distance is calculated by finding the offset to the next access, capped at a maximum value.
- 4) **Training:** The neural network learns the reuse distance mapping and exports to ONNX alongside the normalization statistics.
- 5) **Inference:** The C++ replacer evaluates features on-the-fly to execute evictions. The model remains static online; if access patterns drift, we collect new traces and retrain offline.

E. Falling Back Safely

A learned policy that crashes the database is worse than no learned policy at all. `LEARNEDREPLACER` guarantees operational reliability through a quiet fall-through mechanism. Model loading and inference occur within strict `try/catch` blocks. If the ONNX file is missing, incompatible, or throws a runtime exception, the eviction call routes to a shadow LRU implementation using the same metadata. The database always evicts a valid frame, ensuring stability in production and simplifying A/B testing during development.

V. IMPLEMENTATION

We implemented `LEARNEDREPLACER` as a drop-in alternative to BusTub’s existing replacer interface. The implementation comprises roughly 200 lines of C++ for the replacer and benchmark driver, and 150 lines of Python for the training pipeline. This section focuses on the non-obvious design decisions and the integration boundary between the DBMS and the learned model.

A. Integration with BusTub

BusTub’s buffer pool manager interacts with its replacer via three methods: `RecordAccess()` on page reference, `SetEvictable()` on pin/unpin, and `Evict()` when a victim must be chosen. `LEARNEDREPLACER` exposes this same interface, requiring no structural changes to the broader system. Internally, the replacer updates a per-frame metadata table (frequency, time since last access, average inter-access interval, access type, and workload type) on every `RecordAccess()` call. Because this feature computation is trivial which only requires basic arithmetic and a map lookup, therefore it adds negligible overhead to page accesses. The expensive model inference is strictly deferred to `Evict()`, ensuring `RecordAccess()` never calls into ONNX Runtime.

B. ONNX Inference Path

At startup, the replacer attempts to load the trained model from disk via `Ort::Session`. If loading fails (e.g., missing file, version mismatch), the replacer gracefully degrades to a standard LRU fallback utilizing the same metadata table, requiring no additional state management. During `Evict()`, the replacer constructs a five-element feature vector for each evictable frame, normalizes it, wraps it as an `Ort::Value` tensor of shape `[1, 5]`, and runs a forward pass. The output is the predicted reuse distance; the frame with the highest value is evicted. A critical integration challenge is the normalization handoff. PyTorch’s `torch.onnx.export` does not embed training-time preprocessing logic. Consequently, we export the per-feature mean and standard deviation as a JSON sidecar and hardcode them as compile-time constants in the C++ inference function. Any drift between training and inference statistics silently corrupts predictions, an issue we discuss further in Section VII.

C. Trace Collection and Training Pipeline

Trace collection leverages the same C++ codebase: by intentionally bypassing the model load, the LRU fallback drives eviction while `RecordAccess()` logs page references to a CSV. The Python training pipeline reads these traces, computes rolling features, generates reuse-distance labels via trace lookahead, and trains a PyTorch feedforward network. A subtle but critical requirement is that features must be computed *per workload* before concatenation. Because each workload’s trace independently restarts at timestamp 1, a naive global sort by timestamp erroneously interleaves histories across independent workloads. Computing features within isolated workload traces prevents this data contamination (diagnosed further in Section VI). After training, the model is exported as a self-contained ONNX binary with external data inlined, requiring no additional artifacts at inference time.

D. Build and Deployment

ONNX Runtime is integrated as a single CMake dependency linked against the `bustub_buffer` target. The exported model is read at runtime from the database’s working directory, allowing zero-compilation model updates simply by replacing the file and restarting the database. Ultimately, the integration required only two new source files and one CMake modification, leaving the rest of BusTub untouched.

VI. EVALUATION

We evaluate LEARNEDREPLACER along two dimensions. Section VI-B compares the final trained model against an LRU baseline across three synthetic workloads. Section VI-C details the four-iteration development process, highlighting failure modes and diagnostic steps that are highly relevant to practitioners deploying learned models in C++ systems.

A. Experimental Setup

Experiments use a 64-frame buffer pool with 200 total pages, running 10,000 operations per workload. The benchmark driver simulates page accesses without disk I/O to isolate the policy’s hit-rate performance. The model is a feedforward network (5 inputs $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1 output) trained for 40 epochs with Huber loss and the Adam optimizer. Inference runs via ONNX Runtime 1.25.0 on an Apple M-series CPU.

We evaluate against three workloads:

- **Sequential scan:** Pages 0–199 accessed in repeating order. The working set exceeds cache capacity, representing LRU’s worst-case scenario.
- **Random access:** Uniform random selection over 200 pages. The theoretical hit-rate ceiling is $64/200 = 32\%$.
- **Mixed (80/20):** 80% of accesses hit a 20-page hot set; 20% are uniform random. This workload is highly favorable to recency-based policies.

B. Main Results

Table I compares the final LEARNEDREPLACER (V4) against the LRU baseline. The learned model matches or

TABLE I
FINAL HIT RATES OF LEARNEDREPLACER (V4) VERSUS LRU BASELINE.

Workload	LRU	LearnedReplacer	Δ	Optimum
Sequential	0.00%	26.48%	+26.48 pp	30.87%
Random	31.58%	32.49%	+0.91 pp	32.00%
Mixed (80/20)	84.34%	84.56%	+0.22 pp	$\sim 100\%$

improves upon LRU across all workloads, with dramatic gains on sequential scans.

The sequential scan result is the most significant. LRU achieves exactly 0% due to sequential flooding. The learned model achieves 26.48%, capturing 86% of the analytical optimum (30.87%). It achieves this by correctly predicting that already-resident pages will be re-accessed sooner than incoming pages from later in the scan.

On random and mixed workloads, the learned model performs comparably to LRU. For random access, both hover near the 32% theoretical ceiling, as the lack of temporal locality precludes major optimizations. For the mixed workload, LRU is already near-optimal since the hot set fits entirely in cache; the learned model successfully matches this baseline. Overall, LEARNEDREPLACER matches LRU where it excels and vastly outperforms it where it fails, indicating the model successfully differentiates workload patterns rather than defaulting to a compromised global heuristic.

C. Iterative Development

The V4 model evolved through four iterations, driven by failure modes that only became visible during C++ integration. Table II summarizes this progression.

TABLE II
HIT RATE PROGRESSION ACROSS FOUR ITERATIONS OF LEARNEDREPLACER.

Version	Key change	Seq.	Rand.	Mixed
LRU	Baseline	0.00	31.58	84.34
V1	Binary classification	26.46	31.76	66.87
V2	+ normalization, weighting	30.87	32.07	13.48
V3	Mixed-only training	26.48	32.36	84.26
V4	Reuse-distance regression	26.48	32.49	84.56

V1 (Binary Classification): Framed eviction as predicting if a frame would be accessed within 50 operations. While it beat LRU on sequential scans, it regressed heavily on mixed workloads. The model suffered from class imbalance, collapsing to predict the majority class. Furthermore, the C++ runtime fed unnormalized features to the model, silently mismatching the standardized training data.

V2 (Normalization & Weighting): We hardcoded normalization constants into C++ and applied class weighting to the loss function. While validation metrics looked healthy and sequential hit rates hit the optimum, the mixed hit rate plummeted to **13.48%**. Runtime C++ logging revealed the model was outputting a constant score of 1.0 for all frames. The root cause was sigmoid saturation: the inference-time

feature distribution, altered by the learned policy’s own cache-fill behavior, drifted too far from the LRU-generated training distribution.

V3 (Mixed-Only Training): Training exclusively on the mixed workload trace restored the hit rate to 84.26%, confirming the V2 failure stemmed from cross-workload interference rather than the classification approach. However, a single, generalizable model was still required.

V4 (Reuse-Distance Regression): We shifted to a regression task predicting continuous reuse distance, utilizing Huber loss and introducing a “workload_type” feature to help the model contextualize access patterns. Crucially, we fixed a severe data-pipeline bug: the training traces were previously being globally sorted by timestamp, incorrectly interleaving independent workload histories and corrupting time-based features. Computing rolling features **per workload** before concatenation resolved this, yielding the final V4 results.

D. Diagnostic Findings

This iteration history yields several takeaways for deploying learned components within systems programming:

- **Runtime introspection is critical.** The V2 saturation failure was entirely invisible to standard offline ML validation metrics. It was only diagnosed by instrumenting the C++ `Evict()` function to log raw, per-frame inference scores during live execution.
- **Training-inference distribution drift (covariate shift).** V2 failed because the learned policy altered the cache access patterns at runtime, producing an inference-time feature distribution the model had never seen during its offline training on LRU-generated traces. This feedback loop is fundamental to learned systems: the policy dictates its own future input distribution, necessitating careful distribution alignment or online retraining.
- **The C++/Python boundary is fragile.** Because ONNX exports do not inherently embed training-time preprocessing logic, transferring normalization constants manually is highly error-prone. A single mismatched float silently corrupts runtime predictions without triggering any execution errors.
- **Data engineering dominates model architecture.** The most significant hit-rate improvements across our iterations came from fixing subtle data pipeline bugs (e.g., the V4 trace interleaving) and achieving proper distribution matching, rather than from tweaking the neural network architecture itself.

VII. DISCUSSION

The iterative development of LEARNEDREPLACER surfaced several findings regarding the deployment of learned policies in systems programming. We believe these insights generalize well beyond our specific implementation.

A. The C++/Python Normalization Boundary

ONNX promises a clean separation between model training and deployment. In practice, this separation is incomplete

because the exported computational graph does not natively include the training-time preprocessing logic (e.g., feature standardization). Consequently, the C++ deployment environment must manually replicate this standardization.

In our system, this required exporting per-feature statistics as a JSON sidecar and hardcoding them into the C++ inference path. While functional, this creates a fragile, invisible coupling. Any upstream change to the training data distribution silently invalidates these C++ constants, resulting in corrupted predictions (our V1 failure mode) without triggering any runtime errors. We view this as a systemic flaw in current ML deployment tooling. For robust systems integration, future frameworks must allow preprocessing transformations to be seamlessly embedded directly into the exported model graph.

B. Distribution Alignment Dominates Model Architecture

Consistent with broader observations in learned-systems research, we found that data engineering and distribution alignment vastly outweighed model architecture in driving performance. The most dramatic hit-rate improvements in our system did not come from adding model capacity, but from fixing a subtle pandas `sort_values` bug that contaminated cross-workload features, and from aligning the training data distribution with the inference environment.

Interestingly, our iterations demonstrated that adding *more* training data actually degraded performance when it introduced cross-workload interference (V2), whereas restricting the dataset recovered it (V3). In systems where a learned policy dictates its own future input distribution (cache-fill behavior), the value of training data depends entirely on its distributional alignment with the inference environment, not its raw volume. Practitioners should heavily bias their effort toward feature pipeline integrity rather than architecture search.

C. Reuse-Distance Regression as Bélady Approximation

Bélady’s optimal algorithm, which evicts the page whose next access is furthest in the future, is traditionally considered unimplementable because it requires an oracle. However, when framed as a *regression* problem, it becomes highly amenable to machine learning.

Because the future reuse distance is a continuous quantity, the eviction decision only requires predicting the *relative ordering* of these distances. A binary classifier (our V1 and V2 models) inherently discards this ordering, treating all pages predicted to be accessed “within N operations” as equivalent. By shifting to reuse-distance regression (V4), the model preserves this critical ordering. Even with an imperfect validation MAE of 14.6 operations, regression provides a sufficient approximation of Bélady’s oracle to capture 86% of the optimal hit rate on sequential scans.

D. Workload Identification via Query Planners

While a model trained on a single workload performs well on that specific pattern, real DBMSs serve highly concurrent, mixed workloads. Our solution of introducing a

`workload_feat` indicator allowed a single model to learn distinct strategies, applying near-LRU behavior where optimal and selective eviction where LRU fails.

In our standalone benchmark, this feature was supplied explicitly. In a production DBMS, this signal can be approximated seamlessly by the query planner. For instance, a sequential scan operator could signal `workload_feat = 0`, while an index lookup signals `workload_feat = 2`. This mirrors the underlying philosophy of DBMIN [8]: a buffer pool manager performs best when it knows what the workload is doing. However, rather than using this metadata to route to hardcoded heuristics like DBMIN, a learned system consumes it as a contextual feature to dynamically infer the optimal eviction target. This represents a highly productive design pattern: leveraging existing query planner metadata to specialize a single, global learned model at runtime.

VIII. LIMITATIONS AND FUTURE WORK

We acknowledge several limitations of this work that represent productive directions for future research.

Comparison Against Stronger Baselines. Our evaluation compares LEARNEDREPLACER against LRU. However, Bus-Tub’s actual default replacer is ARC [14], which is significantly more sophisticated and performs closer to optimal on workloads with mixed temporal locality [14]. A direct comparison against ARC, LRU-K [13], and Bélady’s offline optimal [1] would more accurately position the learned policy relative to the state of the art. Furthermore, implementing Bélady’s offline algorithm directly on our traces would allow us to report our results as a “percentage of optimum” uniformly across all workloads, aligning with standard metrics in cache replacement literature.

Limited Workload Diversity. We evaluate on three synthetic access patterns. Real-world database workloads, such as YCSB [17], contain complex, interleaved access patterns that none of our synthetic workloads fully capture. A preliminary experiment with phase-shifting hot sets (where the 20-page hot set rotates every 2,000 operations) suggested that static learned policies face challenges with rapid distribution shift, resulting in hit rates approximately 4 percentage points below LRU. Adapting to such workloads is a critical next step, possibly through attention-based architectures capable of detecting phase boundaries.

Static Models and Concept Drift. Our model is trained once offline and never updated. In production systems, access patterns drift over time as users, queries, and underlying data evolve. A natural extension is periodic retraining and collecting recent access traces, retraining the model offline, and hot-swapping the ONNX file at runtime. Alternatively, future work could explore true online learning via reinforcement learning formulations [12], where the model incrementally updates its weights during normal operation.

Inference Distribution Drift (Covariate Shift). The training traces used for V4 were generated by the LRU fallback. However, the trained model’s eviction decisions inherently alter the cache-fill behavior, meaning at inference time, the

model encounters a state distribution it never saw during training. This feedback loop is a fundamental challenge for any policy that dictates its own future inputs. Future work should address this via iterative retraining methodologies, similar to the Dataset Aggregation (DAgger) techniques utilized in imitation learning approaches for cache replacement [4].

Experimental Rigor. All hit rates reported in this paper are derived from single runs of each workload. To ensure statistical robustness against the noise inherent to neural network initialization and trace generation, future evaluations will report the mean and standard deviation across multiple random seeds for both workload generation and model training.

IX. CONCLUSION

We presented LEARNEDREPLACER, a learned buffer replacement policy implemented inside the CMU BusTub relational database system. By formulating eviction as a regression problem over reuse distance, our approach directly approximates Bélady’s optimal algorithm. LEARNEDREPLACER captures 86% of the theoretical optimum on sequential scans, where LRU achieves 0% by construction, while successfully matching or improving upon LRU across random access and mixed workloads. The model is trained offline in PyTorch, exported to ONNX, and consulted at runtime in C++ via ONNX Runtime, requiring no Python dependency within the database architecture itself.

This work provides a complete, end-to-end implementation of a learned replacement policy paired with a practical, decoupled deployment pipeline. Beyond the numerical results, we documented a four-iteration development process to highlight the friction of integrating learned components into systems programming. We believe these lessons, specifically regarding the fragility of the C++/Python normalization handoff, the dominance of distribution alignment over model architecture, and the critical diagnostic value of runtime C++ introspection are highly generalizable to practitioners deploying learned models in C++ systems. Reuse-distance regression serves as a clean, highly effective Bélady approximation, and ONNX-based deployment is eminently feasible inside a database execution engine. The remaining gap between our results and Bélady’s absolute optimum, alongside the open challenge of managing inference distribution drift through online retraining, represent rich directions for future research.

REFERENCES

- [1] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [2] Y. Yuan and P. Jin, “Learned buffer replacement for database systems,” in *Proceedings of the International Conference on Data Storage and Data Engineering*, ser. ACM International Conference Proceeding Series. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3528114.3528118>
- [3] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Wang, “Driving cache replacement with ML-based LeCaR,” in *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. Boston, MA: USENIX Association, 2018. [Online]. Available: <https://dl.acm.org/doi/10.5555/3277332.3277335>

- [4] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn, "An imitation learning approach for cache replacement," 2020. [Online]. Available: <https://arxiv.org/abs/2006.16239>
- [5] Y. Wang, Y. Meng, J. Wang, and C. Yang, "Lstm-crp: Algorithm-hardware co-design and implementation of cache replacement policy using long short-term memory," *Big Data and Cognitive Computing*, vol. 8, p. 140, 10 2024.
- [6] CMU Database Group, "BusTub Relational Database Management System," <https://github.com/cmu-db/bustub>, 2019, accessed: 2026-04-29.
- [7] ONNX Community, "Open Neural Network Exchange (ONNX)," <https://onnx.ai/>, 2017, accessed: 2026-04-29.
- [8] H.-T. Chou and D. J. DeWitt, "An evaluation of buffer management strategies for relational database systems," in *Proceedings of the 11th International Conference on Very Large Data Bases (VLDB)*, 1985, pp. 127–141.
- [9] A. Dan and D. Towsley, "An approximate analysis of the LRU and FIFO buffer replacement schemes," in *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS '90. New York, NY, USA: Association for Computing Machinery, 1990, pp. 143–152. [Online]. Available: <https://doi.org/10.1145/98460.98525>
- [10] A. Dan, D. Towsley, and P. S. Yu, "Analysis of the generalized clock buffer replacement scheme for database transaction processing," in *Proceedings of the 1992 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS '92/PERFORMANCE '92. New York, NY, USA: Association for Computing Machinery, 1992, pp. 222–231. [Online]. Available: <https://doi.org/10.1145/149439.133084>
- [11] T. Miura, K. Kourai, and S. Yamaguchi, "Cache replacement based on lstm in the second cache in virtualized environment," in *2020 Eighth International Symposium on Computing and Networking Workshops (CANDARW)*, 2020, pp. 421–424.
- [12] M. A. Souza and H. C. Freitas, "Reinforcement learning-based cache replacement policies for multicore processors," *IEEE Access*, vol. 12, pp. 79 177–79 188, 2024.
- [13] E. J. O'Neil, P. E. O'Neil, and G. Weikum, "The LRU-K page replacement algorithm for database disk buffering," in *ACM SIGMOD International Conference on Management of Data*, 1993.
- [14] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *USENIX Conference on File and Storage Technologies (FAST)*, 2003.
- [15] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan, "Learning cache replacement with CACHEUS," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 2021. [Online]. Available: <https://www.usenix.org/conference/fast21/presentation/rodriguez>
- [16] Z. Song, K. Chen, N. Sarda, D. Altınbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, and R. Gummadi, "HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu>
- [17] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *ACM Symposium on Cloud Computing (SoCC)*, 2010.